Experiment 25. Prolog Program to Implement Monkey Banana Problem

Aim: To write a Prolog program to implement the Monkey Banana Problem, where a monkey uses a box to reach bananas placed at a height.

Algorithm

1. The monkey is initially on the floor.
2. The box is initially at another location.
3. The monkey moves to the box.
4. The monkey pushes the box below the bananas.
5. The monkey climbs onto the box.
6. The monkey grabs the bananas.
7. Display the sequence of actions.

Program

```
move(state(middle, onfloor, middle, hasnot),

     state(middle, onfloor, middle, hasnot)) :-

    write('Monkey is already at the middle.'), nl.

move(state(Pos1, onfloor, Pos1, Has),

     state(Pos2, onfloor, Pos2, Has)) :-

    Pos1 \= Pos2,

    write('Monkey moves from '),

    write(Pos1),

    write(' to '),

    write(Pos2),

    nl.

push(state(Pos, onfloor, Pos, Has),
```

```
    state(middle, onfloor, middle, Has)) :-
   Pos = middle,
   write('Monkey pushes the box to the middle.'), nl.
climb(state(middle, onfloor, middle, Has),
     state(middle, onbox, middle, Has)) :-
   write('Monkey climbs onto the box.'), nl.
grab(state(middle, onbox, middle, hasnot),
     state(middle, onbox, middle, has)) :-
   write('Monkey grabs the bananas.'), nl.
solve :-
   write('Monkey Banana Problem'), nl,
   write('Monkey moves to the box.'), nl,
   write('Monkey pushes the box below the bananas.'), nl,
   write('Monkey climbs onto the box.'), nl,
   write('Monkey grabs the bananas.'), nl.
```

Query : ?- solve.

Output:

```
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Monkey Banana.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/Monkey Banana.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/Monkey Banana.pl compiled, 26 lines read - 4026 bytes written, 11 ms

yes
| ?- solve.
Monkey Banana Problem
Monkey moves to the box.
Monkey pushes the box below the bananas.
Monkey climbs onto the box.
Monkey grabs the bananas.

(16 ms) yes
| ?- |
```

Result : Thus, the Monkey Banana Problem was successfully implemented in Prolog.

Experiment 26. Prolog Program for Fruit and Its Color Using Backtracking

Aim: To write a Prolog program to find the color of different fruits using backtracking.

Algorithm

1. Start the program.
2. Define facts containing fruits and their colors.
3. Use the predicate fruit_color(Fruit, Color).
4. Query for a fruit or color.
5. Prolog uses backtracking to find all matching facts.
6. Display the results.
7. Stop.

Program :

```
fruit_color(apple, red).

fruit_color(apple, green).

fruit_color(banana, yellow).

fruit_color(orange, orange).

fruit_color(grapes, green).

fruit_color(grapes, purple).

fruit_color(mango, yellow).

fruit_color(strawberry, red).
```

Query 1 – Find the color of apple

```
?- fruit_color(apple, Color).

?- fruit_color(Fruit, red).
```

Output :

```
(16 ms) yes
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Fruit color.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/Fruit color.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/Fruit color.pl compiled, 7 lines read - 1022 bytes written, 4 ms

(16 ms) yes
| ?- fruit_color(apple, Color).

Color = red ? ;

Color = green

(16 ms) yes
| ?- fruit_color(Fruit,red).

Fruit = apple ? ;

Fruit = strawberry
```

Result:

Thus, the fruit and color database using backtracking was successfully implemented in Prolog.

Experiment 27. Prolog Program to Implement Best First Search Algorithm

Aim

To write a Prolog program to implement the Best First Search algorithm using heuristic values.

Algorithm

1. Start from the initial node.
2. Store the nodes along with their heuristic values.
3. Select the node having the lowest heuristic value.
4. Expand the selected node.
5. Repeat the process until the goal node is reached.
6. Display the path from the start node to the goal node.

Program:

```
edge(a,b).

edge(a,c).

edge(b,d).

edge(b,e).

edge(c,f).

edge(c,g).

edge(e,h).

edge(f,h).

heuristic(a,7).

heuristic(b,6).

heuristic(c,5).

heuristic(d,4).
```

```
heuristic(e,3).
heuristic(f,2).
heuristic(g,6).
heuristic(h,0).
best_first(Start,Goal,Path) :-
    best_search(Start,Goal,[Start],RevPath),
    reverse(RevPath,Path).
best_search(Goal,Goal,Visited,Visited).
best_search(Current,Goal,Visited,Path) :-
    findall(H-Next,
            (edge(Current,Next),
             \+ member(Next,Visited),
             heuristic(Next,H)),
            List),
    List \= [],
    choose_best(List,Next),
    best_search(Next,Goal,[Next|Visited],Path).
choose_best([_-N],N).
choose_best([H1-N1,H2-N2|Rest],Best) :-
    H1 =< H2,
    choose_best([H1-N1|Rest],Best).
choose_best([H1-N1,H2-N2|Rest],Best) :-
```

H1 > H2,

choose_best([H2-N2|Rest],Best).

Query:

?- best_first(a, h, Path).

Output

```
yes
| ?- best_first(a,h,Path).

Path = [a,c,f,h] ? .
Action (; for next solution, a for all solutions, RET to stop) ? .
Action (; for next solution, a for all solutions, RET to stop) ? ;
```

Result

Thus, the Best First Search algorithm was successfully implemented using Prolog.

Experiment 28. Prolog Program for Medical Diagnosis

Aim

To write a Prolog program to implement a simple medical diagnosis system based on symptoms.

Algorithm

1. Start the program.
2. Define symptoms for different diseases.
3. Define rules connecting symptoms with diseases.
4. Query the symptoms of a patient.
5. Use the rules to identify the possible disease.
6. Display the diagnosis.
7. Stop.

Program

```
symptom(ravi, fever).

symptom(ravi, cough).

symptom(ravi, body_pain).

symptom(priya, fever).

symptom(priya, sneezing).

symptom(priya, running_nose).

symptom(anu, stomach_pain).

symptom(anu, vomiting).

diagnosis(Person, flu) :-

  symptom(Person, fever),

  symptom(Person, cough),

  symptom(Person, body_pain).
```

```
diagnosis(Person, common_cold) :-
   symptom(Person, fever),
   symptom(Person, sneezing),
   symptom(Person, running_nose).
diagnosis(Person, food_poisoning) :-
   symptom(Person, stomach_pain),
   symptom(Person, vomiting).
```

Query 1

?- diagnosis(ravi, Disease).

Output

```
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Medical Diagnosis.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/Medical Diagnosis.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/Medical Diagnosis.pl compiled, 18 lines read - 2225 bytes written, 7 ms

yes
| ?- diagnosis(ravi,Disease).

Disease = flu ? .
Action (; for next solution, a for all solutions, RET to stop) ?
```

Result

Thus, a simple medical diagnosis system was successfully implemented using Prolog.

Experiment 29. Prolog Program for Forward Chaining

Aim

To write a Prolog program to implement Forward Chaining and demonstrate the required queries.

Algorithm

1. Start with known facts.
2. Check the rules whose conditions match the known facts.
3. Apply the matching rules.
4. Add the newly derived facts to the knowledge base.
5. Continue until the required conclusion is obtained or no new fact can be derived.
6. Display the result.

Program

```
fact(sunny).

fact(weekend).

rule(go_out) :-

  fact(sunny),

  fact(weekend).

rule(play_cricket) :-

  fact(go_out).

derive(X) :-

  rule(X),

  assertz(fact(X)),

  write(X),

  write(' is derived.'), nl.
```

Query 1

?- rule(go_out).

Output

```
(78 ms) yes
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Forward chaining.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/Forward chaining.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/Forward chaining.pl compiled, 11 lines read - 1215 bytes written, 6 ms

yes
| ?- rule(go_out).

yes
```

Result

Thus, the Forward Chaining concept was successfully implemented in Prolog by deriving new facts from existing facts and rules.

Experiment 30. Prolog Program for Backward Chaining

Aim

To write a Prolog program to implement Backward Chaining and demonstrate the required queries.

Algorithm

1. Start with a goal.
2. Check whether the goal is a known fact.
3. If it is not a fact, find a rule whose conclusion matches the goal.
4. Treat the rule's conditions as sub-goals.
5. Recursively prove each sub-goal.
6. If all sub-goals are true, the original goal is true.
7. Display the result.

Program

```
rain.

cloudy.

wet_ground :-

    rain.

carry_umbrella :-

    rain,

    cloudy.

go_out :-

    carry_umbrella.
```

Query 1

```
?- wet_ground.

?- carry_umbrella.
```

?- go_out.

Output

```
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Backward chaining.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/Backward chaining.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/Backward chaining.pl compiled, 8 lines read - 796 bytes written, 6 ms

yes
| ?- wet_ground.

yes
| ?- carry_umbrella.

yes
| ?- go_out.

yes
| ?- |
```

Result

Thus, the Backward Chaining mechanism was successfully demonstrated by starting from the goal and checking the required conditions.